

Ứng dụng cấu trúc dữ liệu trong lập trình

Xiao Zhou

Nguồn gốc: Applications of data structures in programming

IOI China candidate team papers / 2000 / Xiao Zhou.pdf

Mục lục

1	Dẫn nhập	2
2	Chọn cấu trúc logic hợp lý	2
2.1	Tận dụng thông tin có thể dùng trực tiếp	2
2.2	Không ghi nhận thông tin vô dụng	4
3	Chọn cấu trúc lưu trữ hợp lý	7
3.1	Dùng cấu trúc lưu trữ tuần tự một cách hợp lý	8
3.2	Dùng cấu trúc lưu trữ liên kết khi cần thiết	8
4	Kết hợp nhiều cấu trúc dữ liệu	10
5	Kết luận	11
A	Phụ chú	12
B	Danh sách chương trình	12
B.1	codes.pas: <i>Hidden Code</i> , dùng cấu trúc đồ thị có hướng	12
B.2	boat.pas: Bài đi thuyền, dùng cấu trúc cây nhị phân	15
B.3	under.pas: <i>Underground City</i> , dùng cấu trúc lưu trữ liên kết	17
B.4	sequence.pas: Dãy nhỏ nhất, kết hợp cấu trúc tuyến tính và đồng nhị phân	22
C	Tài liệu tham khảo	25

Từ khóa. Cấu trúc logic; cấu trúc lưu trữ; tối ưu hóa thuật toán.

Tóm tắt

Cấu trúc dữ liệu là nền tảng của lập trình, nên ảnh hưởng của nó đối với hiệu suất thuật toán hiển nhiên không thể xem nhẹ. Bài viết này bàn về cách chọn cấu trúc dữ liệu hợp lý để tối ưu hóa thuật toán, đồng thời thảo luận một số nguyên tắc và phương pháp lựa chọn cấu trúc dữ liệu.

Trước hết, bài viết phân tích tầm quan trọng của cấu trúc logic của dữ liệu và nêu hai nguyên tắc cơ bản khi lựa chọn cấu trúc logic. Tiếp đó, bài viết so sánh ưu nhược điểm của hai cấu trúc lưu trữ tuần tự và liên kết, rồi bàn về cách chọn cấu trúc lưu trữ. Cuối cùng, từ một góc nhìn khác của việc chọn cấu trúc dữ liệu, bài viết tiếp tục thảo luận cách kết hợp nhiều cấu trúc dữ liệu với nhau. Trong khi bàn phương pháp, bài viết cũng chọn một số đề thi tin học có tính đại diện để phân tích làm ví dụ.

1 Dẫn nhập

“Cấu trúc dữ liệu + thuật toán = chương trình”. Câu nói ấy cho thấy bản chất của lập trình là chọn một cấu trúc dữ liệu thích hợp cho bài toán đã xác định, rồi thiết kế một thuật toán tốt. Vì vậy, cấu trúc dữ liệu giữ vị trí rất quan trọng trong lập trình.

Cấu trúc dữ liệu là tập hợp các phần tử dữ liệu mà giữa chúng tồn tại một hoặc nhiều quan hệ đặc thù. Vì “quan hệ” ở đây chỉ quan hệ logic giữa các phần tử dữ liệu, cấu trúc dữ liệu còn được gọi là cấu trúc logic của dữ liệu. Tương ứng với khái niệm khá trừu tượng ấy, cách biểu diễn cấu trúc dữ liệu trong máy tính được gọi là cấu trúc lưu trữ của dữ liệu.

Lập mô hình toán học cho bài toán, rồi thiết kế thuật toán, viết chương trình và gỡ lỗi cho đến khi chạy đúng, là các bước thông thường khi giải một bài toán tin học. Muốn lập mô hình toán học, trước hết ta phải tìm ra quan hệ giữa các đối tượng trong bài toán, tức xác định cấu trúc logic sẽ dùng. Đồng thời, trong quá trình thiết kế thuật toán và hiện thực chương trình, ta phải xác định cách thực hiện các thao tác trên từng đối tượng; cách thao tác lại phụ thuộc vào cấu trúc lưu trữ được chọn. Do đó, cấu trúc logic và cấu trúc lưu trữ tốt hay xấu sẽ trực tiếp ảnh hưởng tới hiệu suất chương trình.

2 Chọn cấu trúc logic hợp lý

Trong lập trình, chọn cấu trúc logic nghĩa là phân tích quan hệ giữa các phần tử dữ liệu trong đề bài, rồi dựa trên những quan hệ đặc thù ấy để chọn cấu trúc logic thích hợp nhằm mô tả bài toán bằng toán học và tiếp tục giải bài toán. Thực chất, cấu trúc logic dùng phương pháp toán học để mô tả các đối tượng thao tác trong bài toán và quan hệ giữa chúng: trừu tượng hóa đối tượng thao tác thành phần tử toán học, và dùng ngôn ngữ toán học để mô tả các quan hệ phức tạp giữa các đối tượng.

Theo đặc tính khác nhau của quan hệ giữa các phần tử dữ liệu, thường có bốn cấu trúc logic cơ bản: tập hợp, cấu trúc tuyến tính, cấu trúc cây, và cấu trúc đồ thị hay mạng. Trong bốn cấu trúc ấy, ngoại trừ tập hợp, nơi các phần tử chỉ có quan hệ “cùng thuộc một tập”, ba cấu trúc còn lại lần lượt biểu diễn các quan hệ “một-một”, “một-nhiều”, và “nhiều-nhiều”.

Vì vậy, trước khi chọn cấu trúc logic, ta phải phân tích rõ đối tượng thao tác trong đề và quan hệ giữa các đối tượng ấy, rồi dựa vào đặc điểm của các quan hệ để chọn cấu trúc logic hợp lý. Điều này đặc biệt quan trọng trong một số bài toán phức tạp: quan hệ giữa dữ liệu rất rối, nhiều cấu trúc logic khác nhau đều có thể giải bài toán, nhưng hiệu suất thuật toán thu được lại khác nhau rất xa.

Với loại bài toán ấy, ta nên dùng tiêu chuẩn nào để chọn cấu trúc logic? Dưới đây là hai yếu tố cần xét đầy đủ khi chọn cấu trúc logic hợp lý.

2.1 Tận dụng thông tin có thể dùng trực tiếp

Trước hết, “thông tin” ở đây là quan hệ giữa các phần tử.

Với thông tin cần xử lý, đại thể có thể chia thành hai loại: “có thể dùng trực tiếp” và “không thể dùng trực tiếp”. Với thông tin có thể dùng trực tiếp, việc sử dụng rất thuận tiện: chỉ cần lấy ra là dùng được.

Còn với loại không thể dùng trực tiếp, ta cũng có thể thông qua một số cách gián tiếp để biến nó thành thông tin dùng được, nhưng quá trình chuyển hóa ấy hiển nhiên tốn thời gian.

Do đó, điều ta cần là càng nhiều thông tin có thể dùng trực tiếp càng tốt. Loại thông tin này càng nhiều, hiệu suất thuật toán càng cao.

Các cấu trúc logic khác nhau chứa lượng thông tin khác nhau, và độ phức tạp khi thuật toán sử dụng thông tin cũng khác nhau. Vì vậy, để thuật toán tận dụng đầy đủ thông tin có thể dùng trực tiếp, đồng thời tránh lãng phí quá nhiều thời gian trong quá trình chuyển thông tin không dùng trực tiếp thành thông tin dùng được, ta cần dùng một cấu trúc logic hợp lý, sao cho nó chứa nhiều thông tin có thể dùng trực tiếp hơn.

Bài toán 1: *Hidden Code* của IOI 1999.

Mô tả bài toán. Bài toán cho một số mã từ và một văn bản. Cần viết chương trình tìm mọi mục mà văn bản chứa các mã từ ấy, rồi ghép các mục tìm được thành một “đáp án” tối ưu sao cho tổng độ dài các mã từ trong đáp án là lớn nhất. Mỗi mục gồm một mã từ và một dãy phủ của mã từ ấy trong văn bản; chẳng hạn `abcbadc` là một dãy phủ của mã từ `abac`. Độ dài dãy phủ không vượt quá 1000. Đồng thời, đáp án yêu cầu các dãy phủ của các mục không chồng lấn nhau.

Phân tích. Với bài này, một thuật toán cơ bản khá dễ nghĩ ra là: lặp theo vị trí kết thúc của dãy phủ trong văn bản, xét xem những mã từ nào được chứa, tìm tất cả các mục, rồi cuối cùng dùng quy hoạch động để ghép các mục thành đáp án tối ưu.

Tạm thời ta chưa xét các mặt khác của thuật toán, mà trước hết chọn cấu trúc logic cho bài toán.

Nếu dùng cấu trúc tuyến tính, chẳng hạn hàng đợi vòng, thì khi xét mã từ t , phương pháp là: ban đầu để con trỏ p trở tới vị trí kết thúc, sau đó liên tục dịch p về trước để lần lượt tìm các chữ cái của mã từ t từ cuối lên đầu. Ví dụ mã từ là `ABDCAB`; văn bản có thể hình dung như sơ đồ sau, trong đó mũi tên ngoài cùng bên phải là vị trí kết thúc và mỗi mũi tên ứng với một chữ cái được tìm thấy:

Huong di chuyen cua con tro p: tu phai sang trai

A	B	D C	A	B	
C	D A C	B	D C	A D C	D B
					A D C
					C B
					A D

Hình 1-1

Vì đề quy định độ dài dãy phủ của mã từ không vượt quá 1000, mỗi lần xét “có chứa hay không” như vậy có độ phức tạp $O(1000)$.

Trong văn bản, vị trí của hai chữ cái kề nhau trong mã từ t không nhất thiết chỉ có quan hệ kề nhau; ví dụ `D` và `C` trong hình 1-1 có thể kề nhau, nhưng giữa `C` và `A` lại có thể cách hai chữ cái. Thông tin trong cấu trúc tuyến tính chỉ tồn tại giữa hai phần tử kề nhau. Dùng lượng thông tin đơn giản như vậy để tìm một chữ cái nào đó của mã từ rõ ràng không hiệu quả.

Nếu ta dựng một đồ thị có hướng, trong đó đỉnh i , tức vị trí thứ i của văn bản, dùng 52 cung nối tới vị trí xuất hiện cuối cùng trước i của từng chữ cái trong `a..z, A..Z`, thì khi cần tìm chữ cái đứng trước một chữ cái nào đó trong mã từ, ta có thể dùng trực tiếp cạnh đã nối, không cần liệt kê. Có thể xem bài toán là: từ một đỉnh của đồ thị có hướng, tìm một đường đi độ dài $\text{length}(t) - 1$, sao cho các đỉnh đi qua có thứ tự theo các chữ cái trong mã từ t .

C D A C B D C A D C D B A D C C B A D

Mọi vị trí nơi tôi lần xuất hiện gần nhất trước do của từng chú cá.

Hình 1-2

Tính toán cho thấy dùng đồ thị để ghi nhận là hoàn toàn chấp nhận được về không gian: ghi 1000 điểm $\times 52$ cung $\times 4$ byte số nguyên dài, tức khoảng 200 KB. Về thời gian, ta có thể tận dụng việc cách nối cung của vị trí i và $i + 1$ thay đổi rất ít: ở hình 1-2, từ vị trí i sang $i + 1$ chỉ có một cung đổi hướng, tức vị trí $i + 1$ cho một cung trở tới vị trí i . Vì vậy, muốn ghi các cung trong đồ thị, chỉ cần gán toàn bộ hướng cung, rồi đổi một cung trong đó.

Nhờ dùng cấu trúc logic đồ thị, hiệu suất tìm chữ cái tăng mạnh. Độ phức tạp của một lần xét là $O(\text{length}(t))$, tệ nhất là $O(100)$, tức nhanh hơn phương pháp ban đầu khoảng 10 lần. Chương trình kèm theo là `codes.pas`.

Trong ví dụ này, tuy cấu trúc tuyến tính cũng có thể giải bài toán, nhưng do tính đặc thù của phép kiểm tra, thông tin cần trong mỗi lần xét không thể tìm từ các phần tử kế nhau. Việc cấu trúc tuyến tính chỉ có quan hệ giữa các phần tử kế nhau trở thành một nhược điểm rõ rệt. Do đó, thông tin trong cấu trúc tuyến tính của bài toán 1 thuộc loại không thể dùng trực tiếp. Ngược lại, cấu trúc đồ thị vừa khớp với nhu cầu: nối bằng cung tất cả các điểm có thể phát sinh quan hệ, để ta dùng quan hệ cung một cách hiệu quả trong quá trình xét và tìm. Tuy cấu trúc đồ thị phức tạp hơn, nó đã chuyển thông tin không thể dùng trực tiếp thành thông tin có thể dùng trực tiếp; hiệu suất thuật toán tăng lên là điều tự nhiên.

2.2 Không ghi nhận thông tin vô dụng

Từ bài toán 1 ta thấy do cấu trúc đồ thị có lượng thông tin lớn, thông tin trong đó về cơ bản đều “có thể dùng”. Nhưng điều này không có nghĩa ta nhất định phải dùng cấu trúc đồ thị. Trong một số tình huống, thông tin có thể dùng trong cấu trúc đồ thị lại hơi thừa.

Thông tin đều dùng được đương nhiên là tốt, nhưng nếu trong đó có quá nhiều thông tin vô dụng, tức không cần thiết, nó chỉ làm tăng độ phức tạp khi suy nghĩ, phân tích và xử lý bài toán, rồi ngược lại gây bất lợi cho việc giải bài.

Bài toán 2: Bài *Đi thuyền* trong kỳ chọn đội Hồ Nam năm 1997.

Mô tả bài toán. Có N người cần đi thuyền. Mỗi thuyền nhiều nhất chở hai người, và hai người cùng thuyền phải cùng họ hoặc cùng tên. Hỏi cần ít nhất bao nhiêu thuyền.

Phân tích. Nhìn bài này, nhiều người sẽ nghĩ tới cấu trúc dữ liệu đồ thị: xem N người là N đỉnh của một đồ thị vô hướng, và nối cạnh giữa mọi cặp người cùng họ hoặc cùng tên.

Điều kiện dùng ít thuyền nhất nghĩa là cần cho càng nhiều cặp người đi chung một thuyền càng tốt; trong đồ thị, điều đó tương ứng với việc dùng ít cạnh nhất để phủ tất cả các đỉnh. Đây vừa khớp với bài toán kinh điển trong lý thuyết đồ thị: tìm phủ cạnh nhỏ nhất. Thuật toán tương ứng là thuật toán tìm ghép cực đại trên đồ thị tổng quát.

Thuật toán tìm ghép cực đại trên đồ thị tổng quát khá phức tạp, cần dùng cây luân phiên mở rộng, co hoa, v.v., và hiệu suất cũng không cao. Vì vậy, ta cần tìm một phương pháp đơn giản và hiệu quả hơn.

Trước hết, do hai thành phần liên thông bất kỳ của đồ thị độc lập tương đối, tức hai đỉnh của một cạnh ghép bất kỳ đều chỉ thuộc cùng một thành phần liên thông, ta có thể xử lý riêng từng thành phần liên thông mà không ảnh hưởng tới kết quả cuối cùng.

Đồng thời, ta có thể ước lượng cận dưới cho số thuyền s . Với một thành phần liên thông chứa P_i đỉnh, số thuyền tối thiểu hiển nhiên là $\lceil P_i/2 \rceil$. Nếu đồ thị có tổng cộng L thành phần liên thông, thì

$$s = \sum_{i=1}^L \left\lceil \frac{P_i}{2} \right\rceil.$$

Sau nhiều lần thử, ta có thể đưa ra phỏng đoán: số cạnh phủ thực tế cần dùng hoàn toàn bằng cận dưới vừa tính.

Nếu dùng cấu trúc đồ thị để chứng minh phỏng đoán trên, có thể dựa vào hai bước sau:

1. Nếu trong một thành phần liên thông không tồn tại đỉnh bậc 1, thì chắc chắn tồn tại chu trình.
2. Xóa một đỉnh bậc 1 và đỉnh kề của nó, hoặc xóa bất kỳ cạnh nào trong một chu trình, thành phần liên thông vẫn liên thông; tức thành phần liên thông chắc chắn tồn tại cạnh không phải cầu.

Vì phương pháp đồ thị không phải trọng tâm ở đây, chi tiết chứng minh không trình bày thêm. Từ cấu trúc dữ liệu đồ thị, thuật toán thu được là: mỗi lần in ra một cạnh không phải cầu, rồi xóa hai đỉnh của cạnh ấy khỏi đồ thị. Độ phức tạp thời gian là $O(n^3)$: tìm một cạnh không phải cầu mất $O(n^2)$, và thao tác tìm cạnh phủ mất $O(n)$.

Do bị giới hạn bởi cấu trúc đồ thị, độ phức tạp thời gian đã khó giảm tiếp. Vì vậy, nếu muốn tiếp tục tối ưu thuật toán, ta chỉ có thể cân nhắc dùng một cấu trúc logic khác. Ở đây ta nghĩ tới cấu trúc cây nhị phân: cụ thể là chuyển từng thành phần liên thông trong đồ thị thành cây nhị phân, rồi dùng cây nhị phân để giải bài toán.

Trước hết chọn một đỉnh bất kỳ trong thành phần liên thông làm gốc, rồi xác định cách dựng cây:

1. Tìm một điểm j cùng họ với nút gốc i , với j chưa thuộc cây nhị phân, làm con trái của i , rồi lấy j làm gốc để dựng cây con.
2. Tìm một điểm k cùng tên với nút gốc i , với k chưa thuộc cây nhị phân, làm con phải của i , rồi lấy k làm gốc để dựng cây con.

Với một thành phần liên thông như hình 2-1-1, dùng cách dựng cây trên có thể nhận được cây nhị phân như hình 2-1-2, lấy nút 1 làm gốc. Trong hình gốc, đường liền biểu diễn cùng họ, đường đứt biểu diễn cùng tên. Vì hình chỉ là sơ đồ nhỏ, có thể mô tả cây thu được như sau:

$$1 \rightarrow (2, 5), \quad 2 \rightarrow (3, \emptyset), \quad 3 \rightarrow (4, \emptyset), \quad 5 \rightarrow (6, \emptyset), \quad 6 \rightarrow (7, 8).$$

Tiếp theo ta chứng minh cây này chắc chắn chứa mọi đỉnh của thành phần liên thông.

Bổ đề 2.1. Nếu cây nhị phân T chứa một nút p , thì tất cả các điểm cùng họ với p trong thành phần liên thông đều thuộc T .

Chứng minh. Để tiện trình bày, quy ước S là tập đỉnh cùng họ với p ; $lc[p, 0]$ là nút p ; $lc[p, i]$, với $i > 0$, là con trái của $lc[p, i - 1]$. Hiển nhiên $lc[p, i]$ cùng họ với p .

Giả sử tồn tại một điểm q thỏa $q \in S$ nhưng $q \notin T$. Vì S là tập hữu hạn, chắc chắn tồn tại một $lc[p, k]$ không có con trái. Khi đó ta có thể đặt $lc[p, k + 1] = q$, suy ra $q \in T$, mâu thuẫn với giả thiết $q \notin T$. Vậy giả thiết sai và bổ đề được chứng minh.

Từ cách chứng minh bổ đề 2.1, tương tự ta có bổ đề 2.2.

Bổ đề 2.2. Nếu cây nhị phân T chứa một nút p , thì tất cả các điểm cùng tên với p trong thành phần liên thông đều thuộc T .

Định lý 1. Cây nhị phân lấy bất kỳ điểm p nào trong một thành phần liên thông làm gốc chắc chắn chứa mọi đỉnh của thành phần liên thông ấy.

Chứng minh. Theo bổ đề 2.1 và bổ đề 2.2, tất cả các điểm cùng họ hoặc cùng tên với p đều chắc chắn ở trong cây nhị phân, tức mọi điểm có cạnh nối với p trong thành phần liên thông đều ở trong cây. Do giữa hai điểm bất kỳ trong một thành phần liên thông đều tồn tại đường đi, mọi điểm của thành phần liên thông ấy đều thuộc cây nhị phân.

Sau khi chứng minh cây nhị phân chứa mọi đỉnh của thành phần liên thông, ta cần chứng minh phỏng đoán ban đầu, tức định lý sau.

Định lý 2. Với cây nhị phân T_m chứa m nút, số thuyền cần dùng là

$$\text{boat}[m] = \left\lceil \frac{m}{2} \right\rceil, \quad m \in N.$$

Chứng minh.

- (i) Khi $m = 1, 2, 3$, mệnh đề hiển nhiên đúng.
- (ii) Giả sử mệnh đề đúng với $m < k, k > 3$. Khi $m = k$, trước hết tìm một nút có tầng sâu nhất trong cây, và gọi cha của nút ấy là p . Khi đó chỉ có ba tình huống sau.

1. p chỉ có một con. Khi đó xóa p và người con duy nhất của p , T_k trở thành T_{k-2} , nên

$$\text{boat}[k] = \text{boat}[k-2] + 1 = \left\lceil \frac{k-2}{2} \right\rceil + 1 = \left\lceil \frac{k}{2} \right\rceil.$$

2. p có hai con, và p là con trái của cha nó. Khi đó có thể xóa p và con phải của p , rồi đặt con trái của p vào vị trí của p . Tương tự, T_k trở thành T_{k-2} , nên $\text{boat}[k] = \text{boat}[k-2] + 1 = \lceil k/2 \rceil$.
3. p có hai con, và p là con phải của cha nó. Khi đó có thể xóa p và con trái của p , rồi đặt con phải của p vào vị trí của p . Tình huống này rất giống trường hợp 2, nên cũng có $\text{boat}[k] = \text{boat}[k-2] + 1 = \lceil k/2 \rceil$.

Kết hợp ba trường hợp, khi $m = k$ ta có $\text{boat}[k] = \lceil k/2 \rceil$. Cuối cùng, từ (i) và (ii), với mọi $m \in N$, $\text{boat}[m] = \lceil m/2 \rceil$.

Từ chứng minh trên, sau khi chuyển cấu trúc đồ thị của dữ liệu thành cấu trúc cây, ta có thuật toán tìm phương án đi thuyền cho một cây nhị phân:

```

proc try(father: integer; var root: integer; var rest: byte);
{In phương án đi thuyền trong cây con gốc root.
 father=0 nghĩa là root là con trai của cha nó,
 father=1 nghĩa là root là con phải của cha nó,
 rest cho biết sau khi in phương án cho cây con có con lại
 một nút gốc chưa đi thuyền hay không.}
begin
  visit[root] := true; {danh dấu root đã tham}
  tìm một nút j cùng họ với root và chưa tham;
  if j <> n+1 then try(0, j, lrest);
  tìm một nút k cùng tên với root và chưa tham;
  if k <> n+1 then try(1, k, rrest);

```

```

if (lrest=1) xor (rrest=1) then begin
  {root chỉ có một con, trường hợp 1}
  if lrest=1 then print(lrest, root) else print(rrest, root);
  rest := 0;
end
else if (lrest=1) and (rrest=1) then begin
  {root có hai con}
  if father=0 then begin
    print(rrest, root); root := j; {trường hợp 2}
  end
  else begin
    print(lrest, root); root := k; {trường hợp 3}
  end;
  rest := 1;
end
else rest := 1;
end;

```

Đây chỉ là thuật toán in phương án đi thuyền cho một cây nhị phân. Muốn in phương án cho tất cả mọi người, ta cần thêm một vòng lặp để tìm gốc của từng cây nhị phân. Vì mỗi điểm chỉ được thăm một lần, và mỗi lần tìm con trái, con phải đều cần một vòng lặp, độ phức tạp thời gian của thuật toán là $O(n^2)$. Chương trình kèm theo là `boat.pas`.

Cuối cùng, ta phân tích nguyên nhân hai cấu trúc dẫn tới hai thuật toán có độ phức tạp khác nhau. Điểm mấu chốt là: tuy cây nhị phân có cấu trúc tương đối đơn giản, nó đã chứa gần như toàn bộ thông tin hữu dụng. Từ thuật toán tìm phương án đi thuyền, ta thấy mọi cạnh trong cây nhị phân không chỉ đều phát huy tác dụng, mà còn không bị dùng lặp lại; tỷ lệ sử dụng thông tin khá cao.

Khi cấu trúc cây đã đủ, một số thông tin trong cấu trúc đồ thị hiển nhiên trở thành thông tin vô dụng. Những thông tin vô dụng dư thừa ấy làm ta khó phát hiện quy luật khi phân tích bài toán, và cũng khó tìm thuật toán hiệu quả. Cũng như tường trong mê cung, càng nhiều càng khó đi. Thông tin vô dụng chỉ làm nhiều tính quy luật của bài toán, khiến ta khó tìm phương pháp giải hơn.

Tiểu kết. Chọn cấu trúc logic của dữ liệu là một khâu then chốt trong việc xây dựng mô hình toán học, còn thuật toán dùng để giải mô hình toán học ấy. Muốn thuật toán hiệu quả, trước hết phải chọn tốt cấu trúc logic của dữ liệu. Trên đây đã nêu hai điều kiện, hay hai hướng suy nghĩ, khi chọn cấu trúc logic; mục đích chung là nâng cao hiệu quả sử dụng thông tin. Thông tin có thể dùng trực tiếp không cần thao tác trung gian nên hiệu quả sử dụng tự nhiên cao. Không ghi nhận thông tin vô dụng giúp ta tập trung hơn vào việc nghiên cứu và phân tích thông tin hữu dụng, và việc sử dụng thông tin cũng nhờ đó được tối ưu hơn.

Tóm lại, trong quá trình giải bài toán, chọn cấu trúc logic hợp lý là một khâu rất quan trọng.

3 Chọn cấu trúc lưu trữ hợp lý

Cấu trúc lưu trữ của dữ liệu được chia thành cấu trúc lưu trữ tuần tự và cấu trúc lưu trữ liên kết. Đặc điểm của cấu trúc lưu trữ tuần tự là dùng vị trí tương đối của phần tử trong bộ nhớ để biểu diễn quan hệ logic giữa các phần tử dữ liệu. Cấu trúc lưu trữ liên kết thì dùng con trỏ chỉ tới địa chỉ lưu trữ của phần tử để biểu diễn quan hệ logic giữa các phần tử dữ liệu.

Do hai cấu trúc lưu trữ khác nhau, khi sử dụng cụ thể chúng cũng có ưu điểm và nhược điểm riêng.

Xét một ví dụ đơn giản: cần ghi một ma trận $n \times n$, trong đó có m phần tử khác 0. Nếu dùng cấu trúc lưu trữ tuần tự, ta sẽ dùng một mảng hai chiều $n \times n$, ghi lại toàn bộ phần tử dữ liệu. Nếu dùng cấu trúc lưu trữ liên kết, ta cần một danh sách liên kết gồm m nút, ghi lại m phần tử dữ liệu khác 0. Từ hai cách ghi khác nhau ấy, ta có thể phân tích ưu nhược điểm của chúng qua các thao tác khác nhau trên dữ liệu.

1. Truy cập ngẫu nhiên một phần tử bất kỳ trong ma trận. Do cấu trúc tuần tự liên nhau về vị trí vật lý, rất dễ lấy địa chỉ lưu trữ của bất kỳ phần tử nào, độ phức tạp là $O(1)$. Với cấu trúc liên kết, vì không có tính liên nhau về vị trí vật lý, trước hết phải duyệt toàn bộ danh sách để tìm địa chỉ lưu trữ của phần tử cần truy cập, độ phức tạp là $O(m)$. Lúc này dùng cấu trúc tuần tự rõ ràng hiệu quả hơn.
2. Duyệt toàn bộ dữ liệu. Độ phức tạp của hai cấu trúc lưu trữ đối với thao tác này rất rõ: cấu trúc tuần tự là $O(n^2)$, cấu trúc liên kết là $O(m)$. Vì thông thường m nhỏ hơn n^2 rất nhiều, lúc này cấu trúc liên kết hiệu quả hơn nhiều.

Ngoài hai thao tác trên, với các thao tác khác, hai cấu trúc này không có ưu nhược điểm quá rõ rệt. Chẳng hạn, xóa hoặc chèn trên danh sách liên kết có thể được biểu diễn trong cấu trúc tuần tự bằng cách đổi phần tử dữ liệu ở vị trí tương ứng.

Vì hiệu suất của hai cấu trúc lưu trữ đối với các thao tác khác nhau chênh lệch khá lớn, khi xác định cấu trúc lưu trữ, ta phải phân tích cẩn thận nhu cầu thao tác trong thuật toán, rồi chọn hợp lý một cấu trúc có thể phát huy điểm mạnh và tránh điểm yếu.

3.1 Dùng cấu trúc lưu trữ tuần tự một cách hợp lý

Khi làm bài hằng ngày, phần lớn chúng ta dùng cấu trúc lưu trữ tuần tự để lưu dữ liệu. Nguyên nhân một mặt là thao tác trên cấu trúc tuần tự thuận tiện, mặt khác trong quá trình hiện thực chương trình, dùng cấu trúc tuần tự tiện gỡ lỗi và tìm lỗi hơn so với cấu trúc liên kết. Vì vậy, theo thói quen, đa số người cho rằng bài toán nào có thể dùng cấu trúc tuần tự để lưu trữ thì tốt nhất nên dùng cấu trúc lưu trữ tuần tự.

Thực ra, cái “tốt” ấy chỉ là một tiêu chuẩn tương đối, được xây dựng trên hai điều kiện tiên quyết sau:

1. Số nút được lưu bằng cấu trúc liên kết không khác nhiều so với số nút được lưu bằng cấu trúc tuần tự. Trong trường hợp này, vì số nút cần lưu khá gần nhau, dùng cấu trúc liên kết hoàn toàn không thể thể hiện ưu điểm ghi ít nút hơn, và thậm chí còn có thể làm giảm hiệu suất thuật toán do thao tác con trở chậm hơn. Nghiêm trọng hơn, vì bản thân con trở chiếm nhiều không gian, lại có nhiều nút, yêu cầu không gian của thuật toán có thể không được đáp ứng.
2. Đây không phải nút thắt hiệu suất của thuật toán. Vì không phải phần tốn thời gian nhất của thuật toán, việc cải tiến ở đây hiển nhiên không ảnh hưởng lớn tới toàn bộ thuật toán; nếu dùng cấu trúc liên kết, thao tác còn trở nên rườm rà quá mức.

3.2 Dùng cấu trúc lưu trữ liên kết khi cần thiết

Trên đây đã phân tích điều kiện dùng cấu trúc lưu trữ tuần tự. Vấn đề còn lại là khi nào nên dùng cấu trúc lưu trữ liên kết.

Do thao tác con trở trong cấu trúc liên kết quả thật khá rườm rà, tốc độ cũng chậm hơn và gỡ lỗi không thuận tiện, mọi người thường không muốn dùng cấu trúc lưu trữ liên kết. Nhưng đó chỉ là quan điểm thông thường; khi cấu trúc liên kết thật sự cải thiện thuật toán rất lớn, ta vẫn buộc phải cân nhắc.

Bài toán 3: *Underground City* của IOI 1999.

Mô tả bài toán. Đã biết bản đồ của một thành phố, nhưng không cho biết vị trí ban đầu của bạn. Bạn cần thông qua một chuỗi bước di chuyển và thăm dò để xác định vị trí ban đầu. Các ràng buộc của đề là:

1. Không được di chuyển tới ô có tường.
2. Chỉ được thăm dò ô kề với vị trí hiện tại theo bốn hướng.

Dưới hai ràng buộc này, số lần thăm dò, không tính di chuyển, cần càng ít càng tốt.

Phân tích. Vì cấu trúc lưu trữ phải do nhu cầu của thuật toán quyết định, trước hết ta xác định thuật toán cho bài toán.

Sau khi phân tích bài, ta có tư tưởng cơ bản: ban đầu giả sử mọi ô không có tường đều có thể là vị trí xuất phát, rồi thông qua thăm dò để từng bước thu hẹp phạm vi vị trí xuất phát, cuối cùng nhận được vị trí xuất phát thật. Đồng thời, để nâng hiệu suất thuật toán, ta còn dùng tư tưởng chia để trị, sao cho mỗi lần thăm dò đều cố gắng thu hẹp phạm vi vị trí xuất phát nhiều nhất có thể, tức để chương trình phụ thuộc vào may mắn ít nhất.

Tiếp đó, ta xác định cấu trúc lưu trữ cho bài này.

Vì bản đồ trong bài là một ma trận hai chiều, thông thường dùng cấu trúc lưu trữ tuần tự là điều tự nhiên. Nhưng trong bài này, cấu trúc lưu trữ tuần tự bộc lộ nhược điểm lớn. Các thao tác ta thực hiện nhiều nhất là lọc phạm vi vị trí xuất phát và quyết định chọn vị trí nào để thăm dò. Dữ liệu cần dùng cho hai thao tác ấy chỉ là một phần rất nhỏ của bản đồ lớn. Nếu dùng cấu trúc lưu trữ tuần tự, dù cần dùng bao nhiêu dữ liệu, ta vẫn luôn phải duyệt toàn bộ bản đồ.

Hình 3-1: dung mang hai chiều; phân chia danh sách chiếm ít ô, nhưng vẫn phải quét cả bản đồ.

Hình 3-2: dung danh sách liên kết:

```
head -> (2,2) -> (2,3) -> (3,4) -> (4,1) -> nil
```

Nếu dùng cấu trúc lưu trữ liên kết như danh sách ở hình 3-2, cần bao nhiêu dữ liệu thì chỉ duyệt bấy nhiêu dữ liệu. Cách này không chỉ phát huy đầy đủ ưu điểm của cấu trúc liên kết, mà vì không cần lấy riêng một dữ liệu nào, mỗi lần đều xét toàn bộ dữ liệu trong danh sách, nó cũng tránh được nhược điểm lớn nhất của cấu trúc liên kết.

Dùng cấu trúc lưu trữ liên kết tuy không làm giảm độ phức tạp thời gian của bài toán, vì trong trường hợp xấu nhất lượng lưu trữ của cấu trúc liên kết gần như bằng cấu trúc tuần tự, nhưng do thể hiện nguyên tắc phát huy điểm mạnh và tránh điểm yếu khi chọn cấu trúc lưu trữ, hiệu suất thuật toán vẫn tăng rất lớn. Thời gian chạy trên các bộ dữ liệu khác nhau như bảng 3-3.

Bộ dữ liệu	Cấu trúc tuần tự	Cấu trúc liên kết
1	0.06s	0.02s
2	1.73s	0.07s
3	1.14s	0.06s
4	3.86s	0.14s
5	32.84s	0.21s
6	141.16s	0.23s
7	0.91s	0.12s
8	6.92s	0.29s
9	6.10s	0.23s
10	17.41s	0.20s

Chương trình dùng cấu trúc lưu trữ liên kết kèm theo là `under.pas`.

Ta chọn cấu trúc lưu trữ liên kết tuy thao tác có thể phức tạp hơn đôi chút, nhưng vì cải tiến đúng nút thắt của thuật toán, hiệu suất thuật toán đã khác hẳn. Qua đó có thể thấy, hiệu quả của việc chọn cấu trúc liên kết khi cần thiết là không thể xem nhẹ.

Tiểu kết. Chọn cấu trúc logic hợp lý liên quan tới việc xây dựng mô hình toán học, nên có lẽ mọi người đều chú ý hơn. Nhưng việc chọn cấu trúc lưu trữ lại dễ bị bỏ qua vì không làm thay đổi độ phức tạp của thuật toán. Vậy một phương pháp không làm giảm độ phức tạp thuật toán có đáng coi trọng không?

Ai cũng biết, cắt tỉa là một phương pháp tối ưu hóa thuật toán thường dùng và được sử dụng rộng rãi, nhưng cắt tỉa cũng không thể thay đổi độ phức tạp của thuật toán. Vì vậy, việc chọn hợp lý cấu trúc lưu trữ, có tác dụng tương tự cắt tỉa, cũng rất đáng coi trọng.

Tóm lại, trong quá trình thiết kế thuật toán, ta phải xét đầy đủ các ảnh hưởng khác nhau do cấu trúc lưu trữ mang lại, rồi chọn cấu trúc lưu trữ hợp lý nhất.

4 Kết hợp nhiều cấu trúc dữ liệu

Những phần trên đều thảo luận cách chọn cấu trúc dữ liệu, gồm lựa chọn cấu trúc logic và lựa chọn cấu trúc lưu trữ; đây là một phương pháp tối ưu hóa thuật toán có tính phổ biến khá lớn. Với phần lớn bài toán, ta đều có thể thông qua việc chọn một cấu trúc logic và một cấu trúc lưu trữ hợp lý để đạt mục đích tối ưu thuật toán.

Nhưng một số bài toán lại thường không thuận như vậy. Khi chọn cấu trúc dữ liệu cho loại bài toán này, ta thường được cái này mất cái kia; thậm chí có khi hoàn toàn không tồn tại một cấu trúc dữ liệu thích hợp. Lúc ấy ta không thể chọn ra một cấu trúc dữ liệu đơn lẻ phù hợp, và các phương pháp trên không còn thích hợp lắm.

Để giải quyết khó khăn trong việc chọn cấu trúc dữ liệu, ta có thể dùng phương pháp kết hợp nhiều cấu trúc dữ liệu. Thông qua kết hợp nhiều cấu trúc dữ liệu, ta đạt được tác dụng bù trừ ưu nhược điểm, để các cấu trúc dữ liệu khác nhau phát huy ưu thế riêng trong thuật toán.

Đó chỉ là tư tưởng chung khi kết hợp nhiều cấu trúc dữ liệu. Cụ thể kết hợp ra sao, ta xem ví dụ sau.

Bài toán 4: Bài *Dãy nhỏ nhất* trong kỳ thi tỉnh Quảng Đông năm 1997.

Mô tả bài toán. Cho một ma trận số nguyên dương $N \times N$, với $N \leq 100$. Cần tìm một đường đi từ vị trí đầu đến vị trí cuối trong ma trận, có thể đi theo bốn hướng lên, xuống, trái, phải, sao cho tổng trị tuyệt đối của hiệu giữa hai số kề nhau trên đường đi là nhỏ nhất.

Phân tích. Thuật toán cơ bản của bài này rất đơn giản: chỉ cần dùng thuật toán Dijkstra để tìm đường đi ngắn nhất từ vị trí đầu đến vị trí cuối. Nhưng ở đây có một vấn đề lớn: $N \leq 100$, tức số điểm trong đồ thị có thể lên tới 10000. Khi đó thuật toán Dijkstra độ phức tạp $O(n^2)$ tỏ ra hơi quá sức.

Ta tiếp tục phân tích thuật toán. Do Dijkstra thường dùng cấu trúc mảng tuyến tính, mỗi lần tìm đường đi ngắn tiếp theo có hai bước:

1. Tìm một đỉnh i không thuộc tập đỉnh xuất phát của các đường đi ngắn nhất, và có khoảng cách tới đích nhỏ nhất. Độ phức tạp của bước này hiển nhiên là $O(n)$.
2. Sửa độ dài đường đi từ các đỉnh kề với i tới đích. Vì nhiều nhất chỉ có 4 điểm, tương ứng bốn hướng, kề với i , độ phức tạp của bước này là $O(1)$, tức hằng số.

Trong hai bước này, tuy bước thứ hai nhiều nhất chỉ đổi độ dài đường đi tới 4 điểm, bước thứ nhất vẫn phải liệt kê tất cả phần tử của mảng, rõ ràng rất tốn thời gian. Để cải tiến bước thứ nhất, ta có thể nghĩ tới cấu trúc đồng nhị phân trong cấu trúc cây. Ưu điểm của đồng là: nút gốc chính là nút tối ưu, và sau khi đổi giá trị của một nút trong đồng, chỉ cần độ phức tạp $O(\log_2 n)$ để hoàn tất quá trình vun đồng. Quá trình này có thể chia thành vun từ dưới lên hoặc từ trên xuống trong cây nhị phân, và độ phức tạp đều như nhau. Nhờ đó, sau khi cấu trúc đồng thay đổi, vun đồng vẫn giữ được tính chất của đồng.

Nếu dùng cấu trúc đồng nhị phân để lưu khoảng cách, bước thứ nhất chỉ cần lấy nút gốc ra, dùng một nút khác trong đồng thay thế rồi vun từ trên xuống một lần. Vì thế độ phức tạp của bước thứ nhất có thể giảm xuống $O(\log_2 n)$. Nhưng lúc này cấu trúc đồng bộc lộ nhược điểm lớn ở bước thứ hai: ta không biết trước vị trí của các điểm kề với i trong đồng. Nếu duyệt đồng để tìm các điểm kề với i , độ phức tạp của bước thứ hai lại trở thành $O(n)$.

Từ hai cấu trúc dữ liệu này, không khó thấy quy luật: dùng mảng tuyến tính thì dễ thực hiện bước thứ hai; dùng đồng nhị phân của cấu trúc cây thì bước thứ nhất có hiệu quả lý tưởng hơn.

Vậy ta có thể kết hợp hai cấu trúc dữ liệu ấy, lấy dài bù ngắn không? Câu trả lời là có.

Ta có thể dùng phương pháp ánh xạ để thiết lập tương ứng một-một giữa phần tử trong cấu trúc tuyến tính và nút trong đồng. Nếu phần tử trong mảng tuyến tính thay đổi, nút tương ứng trong đồng cũng thay đổi; nếu nút trong đồng thay đổi, phần tử tương ứng trong mảng cũng đổi theo.

Sau khi kết hợp hai cấu trúc, dù là bước thứ nhất hay thứ hai, ta đều không cần duyệt tất cả phần tử, mà chỉ cần thực hiện một số hằng định lần thao tác vun đồng độ phức tạp $O(\log_2 n)$. Như vậy, độ phức tạp thời gian toàn bộ trở thành $O(n \log_2 n)$, hiệu suất thuật toán rõ ràng được nâng cao rất nhiều.

Ví dụ này chỉ phân tích sự kết hợp của cấu trúc logic. Với cấu trúc lưu trữ, thực ra cũng cùng một đạo lý. Ta cũng có thể thiết lập quan hệ tương ứng giữa cấu trúc lưu trữ liên kết và cấu trúc lưu trữ tuần tự, đồng thời thực hiện song song thao tác chèn, xóa trong cấu trúc liên kết với việc thay đổi dấu hiệu phần tử dữ liệu trong cấu trúc tuần tự. Như vậy, khi truy cập phần tử dữ liệu, nếu cần duyệt thì dùng cấu trúc liên kết; nếu chỉ cần xét một phần tử nào đó, thì truy cập trực tiếp dấu hiệu của nó trong cấu trúc tuần tự. Thông qua kết hợp cấu trúc lưu trữ, ta phát huy được đầy đủ ưu điểm của cả hai cấu trúc.

Từ đó, ta lại có thêm một phương pháp chọn và sử dụng cấu trúc dữ liệu khi ưu khuyết điểm của các cấu trúc khó phân biệt và khó lựa chọn: dùng ánh xạ để kết hợp các cấu trúc dữ liệu. Đây rõ ràng là một phương pháp và kỹ xảo nữa trong việc sử dụng cấu trúc dữ liệu.

5 Kết luận

Khi dùng cấu trúc dữ liệu hằng ngày, ta thường chỉ xem nó là công cụ để xây dựng mô hình và hiện thực thuật toán, mà chưa xét ảnh hưởng của công cụ ấy đối với hiệu suất chương trình. Khi độ khó của các bài toán tin học ngày càng tăng, yêu cầu về hiệu suất thời gian và không gian của thuật toán cũng ngày càng cao; mà hiệu suất thời gian và không gian của thuật toán, ở mức rất lớn, đều bị cấu trúc dữ liệu chi phối.

Vì vậy, cấu trúc dữ liệu là nền tảng của thi tin học và cũng là nội dung mọi người quen thuộc nhất, nhưng nó giữ vai trò then chốt đối với cả thời gian lẫn không gian của thuật toán. Chỉ khi phân tích ở mức nguyên tắc các phương pháp dùng cấu trúc dữ liệu để tối ưu thuật toán, và khi làm bài xét đầy đủ nguyên tắc lựa chọn phát huy điểm mạnh, tránh điểm yếu cùng phương pháp kết hợp lấy dài bù ngắn, ta mới có thể cải tiến thuật toán hiệu quả hơn.

Cùng với sự phát triển không ngừng của cấu trúc dữ liệu, nhận thức của con người về cấu trúc dữ liệu cũng ngày càng sâu sắc. Bài viết này chỉ thảo luận sơ bộ về cấu trúc dữ liệu; muốn hiểu và nắm vững cấu trúc dữ liệu một cách toàn diện, ta còn cần phân tích và tổng kết sâu hơn. Như câu nói “nuôi quân nghìn ngày, dùng quân một buổi”, chỉ khi ngày thường xây nền tảng tốt và tổng kết kinh nghiệm, ta mới có thể vận dụng thành thạo trong các kỳ thi tin học.

A Phụ chú

1. Định nghĩa cấu trúc dữ liệu trong bài viết này dùng định nghĩa trong sách *Cấu trúc dữ liệu, bản thứ hai*; trong các sách khác, định nghĩa cấu trúc dữ liệu có thể khác.
2. Vì trọng tâm bài viết là cấu trúc dữ liệu chứ không phải thuật toán, khi phân tích bài toán, bài viết có thể chưa trình bày thật chi tiết quá trình suy ra thuật toán của từng bài.
3. Khi so sánh chương trình dùng cấu trúc lưu trữ tuần tự và cấu trúc lưu trữ liên kết trong bài *Underground City*, dữ liệu được chọn là dữ liệu chuẩn của IOI 1999. Máy thử nghiệm là 6x86 200MHz.

B Danh sách chương trình

Vì bài viết thảo luận phương pháp chọn cấu trúc dữ liệu, khi phân tích ví dụ phần lớn dùng cách so sánh các cấu trúc dữ liệu khác nhau. Dưới đây chỉ đưa ra các chương trình dùng cấu trúc dữ liệu tốt hơn.

B.1 codes.pas: *Hidden Code*, dùng cấu trúc đồ thị có hướng

Để nâng hiệu suất chương trình, trong chương trình còn có một số biện pháp tối ưu khác, như sắp xếp mã từ và lưu chọn lọc các mục tìm được. Định dạng vào ra theo chuẩn của đề thi.

```
{ $A+,B-,D+,E+,F-,G-,I-,L+,N-,O-,P-,Q-,R-,S-,T-,V+,X+,Y+ }
{ $M 2048,0,655360 }
const stl1='words.inp'; {ten tep ghi cac ma tu}
      stl2='text.inp'; {ten tep ghi van ban}
      st2='codes.out'; {ten tep xuất}
      max=1023; {so chu cai toi da ghi bang hang doi vong}
type arr1=array[0..10000] of longint;
      arr2=array[0..10000] of shortint;
      arr3=array['A'..'z'] of longint;
var i,j,now:longint; {now danh dau da doc den vi tri thu now cua van ban}
    m,n:integer; {n la so ma tu, m la so muc tim duoc}
    buf:array[1..4095] of char; {bo dem tep vao}
    t:array[1..100] of string[100]; {t[i] ghi tung ma tu}
    last:array[0..max] of ^arr3;
    {vi tri xuất hiện cuối cùng của chu c trước vi tri i được ghi
     la last[i and max,c]}
    nearest:arr3; {nearest[c] la vi tri xuất hiện cuối cùng của c
                  trước vi tri now hiện tại}
    bb,ee:array['A'..'z'] of byte;
    {sau khi sắp xếp, các ma tu có chu cuối c nằm ở bb[c]..ee[c];
     bb[c]>ee[c] nghĩa là không có ma tu có chu cuối c}
    no:array[1..100] of byte;
    {no[i] la vi tri trong danh sách gốc của ma tu thu i sau sắp xếp}
    ss,tt:^arr1;
    nn:^arr2;
    {ss[i],tt[i],nn[i] lần lượt ghi vi tri đầu, vi tri cuối của dãy phụ
     trong mục thu i và số hiệu ma tu tương ứng trong danh sách đã sắp xếp}
```

```

    f:text;
    b:char;

procedure readcod; {doc danh sach ma tu va sap xep}
var f:text;
    i,j,k,l:integer;
    st:string;
begin
    assign(f,st11); reset(f);
    readln(f,n);
    for i:=1 to n do begin
        readln(f,t[i]); no[i]:=i;
    end;
    fillchar(bb,sizeof(bb),1);
    fillchar(ee,sizeof(ee),0);
    {ban dau de moi bb[c]>ee[c], sau khi tim chu c moi sua bb[c], ee[c]}
    for i:=1 to n do begin
        k:=i;
        for j:=i+1 to n do
            {sap theo chu cuoi; neu chu cuoi bang nhau thi ma tu dai hon truoc}
            if (t[j,length(t[j])]<t[k,length(t[k])]) or
                (t[j,length(t[j])]=t[k,length(t[k])]) and
                (length(t[j])>length(t[k])) then k:=j;
        if i<>k then begin
            st:=t[i]; t[i]:=t[k]; t[k]:=st;
            l:=no[i]; no[i]:=no[k]; no[k]:=l;
        end;
        {xac dinh doan trong danh sach moi ung voi tung chu cuoi}
        if t[i,length(t[i])]=b then inc(ee[b])
        else begin
            b:=t[i,length(t[i])]; bb[b]:=i; ee[b]:=i;
        end;
    end;
    close(f);
end;

procedure find(st,ed:longint; e:char);
{tim cac muc da biet vi tri ket thuc la ed, vi tri dau khong nho hon st,
va ma tu chua trong do co chu cuoi e}
var i,k,l:integer;
    j:longint;
    can:boolean;
begin
    for k:=bb[e] to ee[e] do begin
        j:=ed; i:=length(t[k])-1;
        while (i>0) and (j>=st) do begin
            j:=last[j and max]^t[k,i]; dec(i);
        end;
    end;
end;

```

```

{lan luot tim cac chu cai cua ma tu}
if j>=st then begin
  l:=m; can:=true;
  while (l>0) and (tt^[l]>=j) do begin
    if (ss^[l]>=j) and
      (length(t[nn^[l]])>=length(t[k])) then begin
      can:=false; break;
    end;
    dec(l);
  end;
  {chon loc cac muc; can=false nghĩa là muc moi kem muc da co}
  if can then begin
    inc(m); ss^[m]:=j; tt^[m]:=ed; nn^[m]:=k;
  end;
end;
end;
end;

procedure findanswer; {tim dap an toi uu}
var i,j,k:integer;
f:text;
len,next:^arr1;
{len[i] là tổng độ dài mã tu trong đáp án tối ưu xét trên i mục đầu
(không nhất thiết chọn mục i)}
{len[i]=max(len[j])+độ dài mã tu của mục i
(i<j<=m, mục i và mục j không có phần chung)}
{next[i] ghi lại chọn j trong công thức trên}
begin
  new(next); next^[0]:=0;
  new(len); len^[0]:=0;
  k:=0;
  for i:=1 to m do begin
    len^[i]:=len^[i-1]; {không chọn mục i}
    {tính đáp án tối ưu nếu chọn mục i}
    j:=i-1;
    while (j>0) and (tt^[j]>=ss^[i]) do dec(j);
    if len^[j]+length(t[nn^[i]])>len^[i] then begin
      {xét xem chọn mục i có tốt hơn không}
      len^[i]:=len^[j]+length(t[nn^[i]]); next^[i]:=j;
    end;
  end;
end;
assign(f,st2); rewrite(f);
writeln(f,len^[m]); {xuất tổng giá trị của đáp án}
k:=m; {tim cac muc co trong dap an}
while k<>0 do begin
  while len^[k]=len^[k-1] do dec(k); {bang nhau thì không chọn mục k}
  writeln(f,no[nn^[k]],' ',ss^[k],' ',tt^[k]);
  k:=next^[k];
end;

```

```

    end;
    close(f);
end;

begin
    readcod;
    new(ss); new(tt); new(nn);
    assign(f,st12);
    setttextbuf(f,buf);
    reset(f);
    fillchar(nearest,sizeof(nearest),0);
    for i:=0 to max do begin
        new(last[i]); fillchar(last[i]^,sizeof(last[i]^),0);
    end;
    {khởi tạo}
    while not seekeoln(f) do begin
        inc(now);
        last[now and max]^:=nearest;
        read(f,b); nearest[b]:=now; {cập nhật lần xuất hiện cuối của b}
        if bb[b]<=ee[b] then {xét các mã tu có chu cuối b}
            if now>=1000 then find(now-999,now,b)
            else find(1,now,b);
        {xác định cần duyệt vì trị đầu max(now-999,1), rồi tìm mục}
    end;
    close(f);
    findanswer;
end.

```

B.2 boat.pas: Bài đi thuyền, dùng cấu trúc cây nhị phân

Định dạng vào của bài này: dòng đầu là tổng số người; mỗi dòng sau là họ và tên của một người, cách nhau bởi một dấu cách. Họ dài không quá 5, tên dài không quá 10. Tập ra mỗi dòng cho biết những người ngồi trên một thuyền; dòng cuối là tổng số thuyền cần dùng.

```

{$A+,$B-,$D+,$E+,$F-,$G-,$I-,$L+,$N-,$O-,$P-,$Q-,$R-,$S-,$T-,$V+,$X+}
{$M 65520,0,655360}
const Maxn=5000; {số người tối đa}
type Fnamearr=array[1..Maxn] of string[5];
    Gnamearr=array[1..Maxn] of string[10];
var Fname:^Fnamearr; {ghi họ của mọi người}
    Gname:^Gnamearr; {ghi tên của mọi người}
    visit:array[1..Maxn] of boolean; {visit[i] cho biết người i đã tham chưa}
    total,n:integer; {total là số thuyền cần dùng, n là tổng số người}
    fo:text;

procedure init; {đọc tên mọi người}
var f:text;
    st,t:string;

```

```

    i,j:integer;
begin
    new(Fname); new(Gname);
    write('Input file name:'); readln(st); assign(f,st); reset(f);
    readln(f,n);
    for i:=1 to n do begin
        readln(f,t);
        j:=pos(' ',t);
        Fname^[i]:=copy(t,1,j-1);
        delete(t,1,j);
        Gname^[i]:=t;
    end;
    close(f);
end;

procedure print(i,j:integer); {xuat viec i va j cung ngoi mot thuyen}
var t:integer;
begin
    t:=17-length(Fname^[i])-length(Gname^[i]);
    writeln(fo,Fname^[i],' ',Gname^[i],':t, '- ',Fname^[j],' ',Gname^[j]);
    inc(total);
end;

procedure try(father:integer; var root:integer; var rest:byte);
{dung cay con goc root va xuat phuong an di thuyen trong cay con}
{father cho biet root la con nao cua cha: 0 la con trai, 1 la con phai}
{rest cho biet sau khi xuat phuong an cho cay con co mot nguoi di rieng khong}
{rest=1 nghia la co, va nguoi di rieng co the doi len vi tri goc root}
var j,k:integer;
    Lrest,Rrest:byte;
    {Lrest, Rrest la gia tri rest cua cay con trai va phai cua root}
begin
    visit[root]:=true;
    j:=1;
    while (j<=n) and (visit[j] or (Fname^[root]<>Fname^[j])) do inc(j);
    {tim mot nguoi j cung ho voi root va chua tung tham}
    Lrest:=0;
    if j<=n then try(0,j,Lrest); {dung cay con goc j va xuat phuong an}
    k:=1;
    while (k<=n) and (visit[k] or (Gname^[root]<>Gname^[k])) do inc(k);
    {tim mot nguoi k cung ten voi root va chua tung tham}
    Rrest:=0;
    if k<=n then try(1,k,Rrest); {dung cay con goc k va xuat phuong an}
    {phan bo thuyen theo viec hai cay con con nguoi du hay khong}
    if (Lrest=1) xor (Rrest=1) then begin {chi mot cay con con nguoi du}
        if Lrest=1 then print(root,j) else print(root,k);
        {dua nguoi con du cung ngoi voi root}
        rest:=0;
    end;
end;

```



```

end
else if (Lrest=1) and (Rrest=1) then begin {ca hai cay con deu con nguoi}
    {dua quan he giua root va cha de chon nguoi di cung root,
    roi doi nguoi con lai len vi tri goc root}
    if father=0 then begin
        print(root,k); root:=j;
    end
    else begin
        print(root,j); root:=k
    end;
    rest:=1;
end
else rest:=1;
end;

procedure findanswer; {tim moi cay trong rung, tuc moi thanh phan lien thong}
var i,j:integer;
    rest:byte;
begin
    assign(fo,'output.txt'); rewrite(fo);
    for j:=1 to n do
        if not visit[j] then begin {con nguoi chua tham thi o mot cay khac}
            i:=j;
            try(1,i,rest);
            {lay diem chua tham lam goc, dung cay va xuat phuong an trong cay}
            if rest=1 then begin {cuoi cung co con lai mot nguoi khong}
                writeln(fo,Fname^[i], ' ',Gname^[i]);
                inc(total);
            end;
        end;
        writeln(fo,'The total of boats is ',total); {xuat tong so thuyen}
    close(fo);
end;

begin
    init;
    findanswer;
end.

```

B.3 under.pas: *Underground City*, dùng cấu trúc lưu trữ liên kết

Định dạng vào ra của chương trình theo chuẩn của đề thi.

```

{$A+,B-,D+,E+,F-,G-,I-,L+,N-,O-,P-,Q-,R-,S-,T-,V+,X+}
{$M 65520,0,655360}
uses undertpu;
const c:array[1..4,1..2] of shortint=((0,-1),(-1,0),(1,0),(0,1));
    {do doi toa do theo bon huong}

```

```

    d:array[1..4] of char=('S','W','E','N'); {chu cai ung voi bon huong}
    stl='under.inp'; {ten tep vao}
type aa=record
    x,y:shortint;
    next:integer;
end;
arr=array[0..10000] of aa;
var a:array[1..100,1..100] of char;
{ban do thanh pho ngam; toa do (x,y) ung voi a[x,y].
a[x,y]='0' nghĩa là không có tường, a[x,y]='W' nghĩa là có tường}
z:array[-100..100,-100..100] of char;
{ban do moi gom thong tin da biet, (0,0) là vị trí xuất phát.
'0','W' lần lượt là không có tường và có tường}
{ngoài ra dùng 'Q' và 'U' cho ô chưa biết; ô 'U' có thể tham dò,
con ô 'Q' thì không}
uk,can:^arr;
{dùng mảng để hiện thực danh sách liên kết; nút đầu là phần tử chỉ số 0}
{kieu nút: x,y là toa do, next là địa chỉ nút kế trong mảng}
{uk ghi các vị trí hiện có thể tham dò nhưng chưa biết trạng thái;
can ghi toa do các điểm xuất phát còn khả năng}
tail,rest,v,u:integer;
{rest là số điểm xuất phát còn khả năng}
{tail là con trỏ cuối của danh sách uk, đồng thời cho biết uk đã dùng
không gian 0..tail-1}
{u và v lần lượt là số cột và số hàng của bản đồ}

procedure readp; {doc ban do thanh pho}
var f:text;
    i,j:integer;
begin
    assign(f,stl); reset(f);
    readln(f,u,v);
    for i:=v downto 1 do begin
        for j:=1 to u do read(f,a[j,i]);
        readln(f);
    end;
    close(f);
end;

procedure main; {thu tuc chinh tim diem xuất phát}
var nowx,nowy,nextx,nexty,o:integer;
    {vị trí hiện tại so với điểm xuất phát là (nowx,nowy),
    mục tiêu tham dò tiếp theo là (nextx,nexty)}
    e:char;

procedure changeUK(xx,yy:integer);
{sau khi thêm ô không có tường (xx,yy),
có thể tham dò các ô chưa biết kế nó}

```

```

var i,j:integer;
begin
  for i:=1 to 4 do
    if z[xx+c[i,1],yy+c[i,2]]='Q' then begin {xet bon o ke}
      z[xx+c[i,1],yy+c[i,2]]:='U'; {dat thanh co the tham do}
      with uk^[tail] do begin {them o co the tham do vao cuoi danh sach}
        x:=xx+c[i,1]; y:=yy+c[i,2];
        next:=tail+1;
      end;
      inc(tail); {de tail tro toi mot vi tri chua dung}
    end;
  end;
end;

```

```

procedure getready; {chuan bi truoc khi tham do}
var i,j:integer;
begin
  nowx:=0; nowy:=0;
  fillchar(z,sizeof(z),'Q');
  z[0,0]:='0'; {ban dau chi biet diem xuat phat khong co tuong}
  new(can); can^[0].next:=1;
  {thong ke moi o co the la diem xuat phat va dua vao danh sach can}
  for i:=1 to u do
    for j:=1 to v do
      if a[i,j]='0' then begin
        inc(rest);
        with can^[rest] do begin
          x:=i; y:=j; next:=rest+1;
        end;
      end;
  can^[rest].next:=0;
  new(uk); tail:=1; uk^[0].next:=1;
  changeUK(0,0); {cac o ke diem xuat phat thanh o co the tham do}
end;

```

```

procedure change; {tim o xac dinh duoc ma khong can tham do}
var px,py,i,j,d:integer;
  {d ghi trang thai: 1 la co tuong, 2 la khong co tuong,
   3 la ca hai deu co the}
begin
  i:=0;
  while uk^[i].next<>tail do begin {duyet o chua biet co the tham do}
    px:=uk^[uk^[i].next].x; py:=uk^[uk^[i].next].y;
    d:=0; j:=0;
    while (d<3) and (can^[j].next<>0) do begin
      j:=can^[j].next;
      if a[can^[j].x+px,can^[j].y+py]='W'
        then d:=d or 1 {ghi kha nang co tuong}
        else d:=d or 2; {ghi kha nang khong co tuong}
    end;
  end;

```

```

end;
if d<3 then begin {neu khong phai ca hai trang thai deu co the}
    uk^[i].next:=uk^[uk^[i].next].next;
    {trang thai o nay da xac dinh, xoa khoi danh sach}
    if d=1 then z[px,py]:='W'; {chi co the la co tuong}
    if d=2 then begin {chi co the la khong co tuong}
        z[px,py]:='0';
        changeUK(px,py); {them cac o ke (px,py) co the tham do}
    end;
end
else i:=uk^[i].next;
end;
end;

procedure find(var xx,yy:integer); {tim vi tri tham do tot nhat}
var min,px,py,d1,d2,i,j,o:integer;
    {min la muc phu thuoc may man; cang nho cang tot}
begin
    min:=maxint; i:=0;
    while uk^[i].next<>tail do begin
        px:=uk^[uk^[i].next].x; py:=uk^[uk^[i].next].y;
        d1:=0; j:=0;
        while can^[j].next<>0 do begin
            j:=can^[j].next;
            if a[can^[j].x+px,can^[j].y+py]='W'
                then inc(d1); {so kha nang co tuong tang 1}
        end;
        d2:=rest-d1; {tinh so kha nang khong co tuong}
        if abs(d1-d2)<min then begin {chon (px,py) neu can bang hon}
            min:=abs(d1-d2); o:=i; xx:=px; yy:=py;
        end;
        i:=uk^[i].next;
    end;
    uk^[o].next:=uk^[uk^[o].next].next; {xoa muc tieu tham do khoi danh sach}
end;

procedure path(x1,y1,x2,y2:integer);
{tim phuong an di chuyen va tham do, roi loc diem xuat phat}
var s,t,k,l,i,j:integer;
    p:arr;
begin
    {dung BFS tim duong tu (x2,y2) den (x1,y1)}
    p[1].x:=x2; p[1].y:=y2; p[1].next:=0; s:=1; t:=1;
    while (p[s].x<>x1) or (p[s].y<>y1) do begin
        for k:=1 to 4 do
            if z[p[s].x+c[k,1],p[s].y+c[k,2]]='0' then begin
                inc(t);
                p[t].x:=p[s].x+c[k,1]; p[t].y:=p[s].y+c[k,2];
            end;
        end;
        s:=t;
    end;
end;

```

```

        z[p[t].x,p[t].y]:='P'; {danh dau vi tri da tim}
        p[t].next:=s; {ghi cha cua nut t}
    end;
    inc(s);
end;
{di chuyen den o ke vi tri can tham do (x2,y2)}
l:=s; j:=p[l].next;
while j<>1 do begin
    for i:=1 to 4 do
        if (p[l].x+c[i,1]=p[j].x) and
            (p[l].y+c[i,2]=p[j].y) then break;
        move(d[i]);
        l:=j; j:=p[l].next;
    end;
    {tim huong tham do}
    nowx:=p[l].x; nowy:=p[l].y;
    for i:=1 to 4 do
        if (p[l].x+c[i,1]=p[1].x) and
            (p[l].y+c[i,2]=p[1].y) then break;
    e:=look(d[i]); {tham do}
    z[x2,y2]:=e; {danh dau tren ban do}
    if e='0' then changeUK(x2,y2); {neu khong co tuong, them vi tri}
    for i:=-u to u do
        for j:=-v to v do
            if z[i,j]='P' then z[i,j]:='0'; {khoi phuc ban do}
        j:=0;
    while can^[j].next<>0 do begin {loc diem xuat phat}
        k:=j;
        j:=can^[j].next;
        if a[can^[j].x+x2,can^[j].y+y2]<>e then begin
            {trang thai cua vi tri tuong doi khop ket qua tham do}
            dec(rest); can^[k].next:=can^[j].next; j:=k;
        end;
    end;
end;

begin
    getready; {chuan bi truoc khi tham do}
    start; {bat dau tham do}
    while rest>1 do begin
        change; {tim o co trang thai xac dinh ma khong can tham do}
        find(nextx,nexty); {tim vi tri tham do tot nhat}
        path(nowx,nowy,nextx,nexty);
        {tim qua trinh di chuyen, tham do cu the va loc diem xuat phat}
    end;
    o:=can^[0].next;
    finish(can^[o].x,can^[o].y); {tim duoc vi tri xuat phat}
end;

```

```

begin
    readp;
    main;
end.

```

B.4 sequence.pas: Dãy nhỏ nhất, kết hợp cấu trúc tuyến tính và đồng nhị phân

Dữ liệu vào của chương trình: dòng đầu là cạnh N của ma trận; tiếp theo là N dòng chứa ma trận số nguyên dương $N \times N$; dòng cuối gồm bốn số, biểu diễn vị trí đầu và vị trí cuối theo thứ tự hàng rồi cột. Tập ra có dòng đầu là giá trị nhỏ nhất của tổng trị tuyệt đối hiệu giữa hai số kề nhau trong dãy, dòng thứ hai là dãy kề nhau tương ứng.

```

{$A+,B-,D+,E+,F-,G-,I+,L+,N-,O-,P-,Q-,R-,S+,T-,V+,X+}
{$M 16384,0,655360}
const b:array[1..4,1..2] of shortint=((1,0),(0,1),(-1,0),(0,-1));
type arr=array[0..101,0..101] of integer;
    aa=record
        {kieu nut trong dong; w la khoang cach den dich,
         (x,y) la vi tri cua nut trong ma tran}
        w:integer;
        x,y:byte;
    end;
var a,d:array[0..101,0..101] of byte;
    {a[i,j] ghi ma tran; d[i,j] la huong buoc tiep theo trong duong ngan nhat
     tu (i,j) den dich}
    pos:^arr;
    {pos[i,j] la mang anh xa giua cac cau truc du lieu, bieu thi vi tri luu
     khoang cach tu (i,j) den dich trong cau truc dong, tuc h[pos[i,j]].w}
    h:array[0..10001] of aa;
    {dong nhai phan bieu dien bang mang; con trai cua h[i] la h[i*2],
     con phai la h[i*2+1]}
    x1,y1,x2,y2,n:byte; {n la canh ma tran; (x1,y1),(x2,y2) la dau va dich}
    t:integer; {tong so nut trong dong}

procedure readp; {doc du lieu}
var f:text;
    st:string;
    i,j:byte;
begin
    new(pos);
    write('File name:'); readln(st); assign(f,st); reset(f);
    readln(f,n);
    for i:=1 to n do
        for j:=1 to n do
            read(f,a[i,j]);
    readln(f,x1,y1,x2,y2);
    close(f);

```

```

end;

procedure swap(i,j:integer); {doi nut thu i va thu j trong dong}
var k:aa;
begin
    k:=h[i]; h[i]:=h[j]; h[j]:=k; {doi khoang cach}
    pos^[h[i].x,h[i].y]:=i; pos^[h[j].x,h[j].y]:=j; {doi vi tri anh xa}
end;

procedure heap1(i:integer); {vun dong tu nut i xuong duoi}
var j:integer;
begin
    while i*2<=t do begin
        if (i*2+1<=t) and (h[i*2+1].w<h[i*2].w)
            {tim con co khoang cach den dich ngan hon}
            then j:=i*2+1
            else j:=i*2;
        if h[j].w<h[i].w then begin {co can doi voi nut goc khong}
            swap(i,j);
            i:=j;
        end
        else break;
    end;
end;

procedure heap2(i:integer); {vun dong tu nut i len tren}
begin
    while (i<>1) and (h[i div 2].w>h[i].w) do begin {co can doi voi cha khong}
        swap(i div 2,i);
        i:=i div 2;
    end;
end;

procedure main; {tim duong ngan nhat tu moi vi tri den dich}
var xx,yy,i,j:byte;
    fo:text;
begin
    h[10001].w:=9999;
    for i:=1 to n do
        for j:=1 to n do
            pos^[i,j]:=10001;
        {dat khoang cach tu moi vi tri den dich la vo cuc}
        h[0].w:=0;
        for i:=1 to n do begin
            pos^[i,0]:=0; pos^[0,i]:=0; pos^[i,n+1]:=0; pos^[n+1,i]:=0;
        end;
        {them mot vong quanh ma tran de khong di ra ngoai, gia su khoang cach
        den vong nay deu la 0}

```

```

{vi 0 không thể nhỏ hơn nửa, sẽ không đi ra ngoài mà tràn}
with h[1] do begin
    w:=0; x:=x2; y:=y2;
end;
pos^[x2,y2]:=1; t:=1; {dua dich lam goc dong}
repeat
    {nut goc trong dong la diem dau cua duong ngan nhat}
    for i:=1 to 4 do begin {doi khoang cach tu cac diem ke goc den dich}
        xx:=h[1].x+b[i,1]; yy:=h[1].y+b[i,2]; {tinh toa do diem ke}
        if h[pos^[xx,yy]].w=9999 then begin {xet da o trong dong chua}
            inc(t);
            with h[t] do begin {them mot nut moi vao dong}
                w:=h[1].w+abs(a[h[1].x,h[1].y]-a[xx,yy]);
                x:=xx; y:=yy;
                pos^[xx,yy]:=t; d[xx,yy]:=i;
            end;
            heap2(t); {vun len tu nut moi them}
        end
        else if h[1].w+abs(a[h[1].x,h[1].y]-a[xx,yy])<h[pos^[xx,yy]].w
            then begin {xet co tim duoc khoang cach ngan hon khong}
                h[pos^[xx,yy]].w:=h[1].w+abs(a[h[1].x,h[1].y]-a[xx,yy]);
                {cap nhat khoang cach}
                heap2(pos^[xx,yy]); {vun len tu diem tuong ung trong dong}
                d[xx,yy]:=i;
            end;
        end;
        swap(1,t); pos^[h[t].x,h[t].y]:=0; dec(t); {lay nut goc ra}
        heap1(1); {vun xuong tu nut goc}
    until (h[1].x=x1) and (h[1].y=y1); {da tim duong tu dau den dich}
    assign(fo,'output.txt'); rewrite(fo);
    writeln(fo,h[1].w); {xuat khoang cach}
    {xuat cac diem tren duong di}
    while (x1<>x2) or (y1<>y2) do begin
        write(fo,a[x1,y1],' '); i:=x1; j:=y1;
        dec(x1,b[d[i,j],1]); dec(y1,b[d[i,j],2]);
    end;
    writeln(fo,a[x2,y2]);
    close(fo);
end;

begin
    readp;
    main;
end.

```


C Tài liệu tham khảo

1. Yan Weimin, Wu Weimin. *Cấu trúc dữ liệu*, bản thứ hai. Nhà xuất bản Đại học Thanh Hoa.
2. Wu Wenhui, Wang Jiande. *Phân tích và thiết kế chương trình cho các thuật toán thực dụng*. Nhà xuất bản Công nghiệp Điện tử.
3. Wu Wenhui, Wang Jiande. *Hướng dẫn thi Olympic Tin học quốc tế và toàn quốc cho thanh thiếu niên: thuật toán đồ thị và lập trình*.
4. *Olympic Tin học*, tạp chí quý, số 1 và số 2 năm 1998.
5. Đề IOI 1999 và các đề thi cấp tỉnh Hồ Nam qua các năm.